

Ниже полная BC_v1, **редакция 6**. В начале UFW выключается (пункт 2), правила — в пункте 13. Один сертификат ACME панели на три имени; nginx берёт те же файлы. Сертификат CDN (cdn.) — Certificate Manager.

BC_v1 — VPN-нода для обхода белых списков

Версия: BC_v1, редакция 6 · Август 2026
Стек: Ubuntu 24.04 LTS · nginx · 3x-ui v3.6.0 · Xray
Транспорт: VLESS + XHTTP (packet-up , аплинк GET) + TLS
Вход клиентов: cdn.vc01zbbxr.tech через Yandex Cloud CDN
Маскировка: кеширующее зеркало Debian на том же nginx
Подписка на ноду: выключена (путь в панели всё равно /oob/)
Нагрузка: ~700 пользователей на ноду

Таблица переменных

Переменная	Значение
NODE_IP	публичный IP ВМ в Yandex Cloud
MIRROR_DOMAIN	vc01zbbxr.tech , www.vc01zbbxr.tech
ORIGIN_DOMAIN	origin.vc01zbbxr.tech
CDN_DOMAIN	cdn.vc01zbbxr.tech
XHTTP_PATH	/healthcheck.api/
SUB_PATH	/oob/ (задаётся в панели, подписка OFF)
PANEL_PORT	8080 (открыт, без ограничения по IP)
XRAY_PORT	10443 (только 127.0.0.1)
UPSTREAM	deb.debian.org
CACHE_DIR / CACHE_MAX	/var/cache/nginx/mirror / 10g
CERT_DIR	/root/cert/vc01zbbxr.tech/ (ACME 3x-ui: fullchain.pem + privkey.pem)

1. Архитектура и порты

```
Клиент (сеть с белыми списками)
| HTTPS → cdn.vc01zbbxr.tech:443
| SNI = cdn.vc01zbbxr.tech, ALPN браузерный (uTLS edge)
| XHTTP packet-up, GET, путь /healthcheck.api/
▼
Yandex Cloud CDN (адреса из yc.json)
| HTTPS → origin.vc01zbbxr.tech:443
| Host = origin.vc01zbbxr.tech
▼
BM: nginx :443
├─ vc01zbbxr.tech, www      → зеркало Debian
└─ origin.vc01zbbxr.tech (default_server)
    /healthcheck.api/  только с адресов CDN → 127.0.0.1:10443
    всё прочее, сканы по IP                → зеркало Debian
▼
Xray → routing → direct или каскад

Панель: https://vc01zbbxr.tech:8080/<секретный_путь>
Ключи выдаются вручную (vless:// или JSON), не через подписку.
```

Порт	Назначение	Наружу
443/tcp	nginx: зеркало + туннель	да
80/tcp	ACME + редирект	да
8080/tcp	панель 3x-ui	да, без ограничения по IP
22/tcp	SSH	да, пароль допустим
10443/tcp	Xray XHTTP	нет

Требования к BM: Ubuntu 24.04 amd64, 2-4 vCPU, 4-8 GB RAM, диск ≥ 20 GB (из них 10 GB кеш), канал ≥ 1 Gbit/s.

2. Подготовка ОС

UFW на время установки **выключить**. Let's Encrypt проверяет домен по **порту 80 с интернета**; включённый UFW с deny incoming даёт Timeout during connect. Порты 22/80/443/8080 откроем явно в пункте 13.

```
sudo ufw disable
sudo ufw status
```

Ожидается Status: inactive. Группу безопасности Yandex Cloud не отключать: входящие TCP 22, 80, 443, 8080 с 0.0.0.0/0 должны быть разрешены уже сейчас.

```
apt update && apt upgrade -y
adduser vpnXX
usermod -aG sudo vpnXX
```

```
sudo nano /etc/ssh/sshd_config
# PermitRootLogin no
sudo sshd -t && sudo systemctl restart ssh
```

Ключ SSH не обязателен. Не закрывайте сессию root, пока не проверите вход под vpnXX.

3. Тюнинг ядра и лимитов

```
sudo nano /etc/sysctl.d/99-vpn.conf
```

```
net.core.somaxconn = 65535
net.core.netdev_max_backlog = 250000
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fastopen = 3
net.ipv4.tcp_keepalive_time = 600
net.ipv4.tcp_keepalive_intvl = 30
net.ipv4.tcp_keepalive_probes = 5
net.netfilter.nf_conntrack_max = 1048576
net.netfilter.nf_conntrack_buckets = 262144
net.netfilter.nf_conntrack_tcp_timeout_established = 3600
net.netfilter.nf_conntrack_tcp_timeout_time_wait = 30
net.core.rmem_max = 33554432
net.core.wmem_max = 33554432
net.ipv4.tcp_rmem = 4096 87380 33554432
net.ipv4.tcp_wmem = 4096 65536 33554432
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
fs.file-max = 2097152
fs.nr_open = 1048576
fs.inotify.max_user_instances = 1024
```

```
sudo modprobe nf_conntrack
sudo sysctl --system
sudo nano /etc/security/limits.d/99-vpn.conf
```

```
*          soft nofile 1048576
*          hard nofile 1048576
root       soft nofile 1048576
root       hard nofile 1048576
www-data   soft nofile 1048576
www-data   hard nofile 1048576
```

4. DNS (до CDN)

Имя	Тип	Значение
vc01zbbxr.tech	A	NODE_IP
www.vc01zbbxr.tech	A	NODE_IP
origin.vc01zbbxr.tech	A	NODE_IP
_acme-challenge.cdn.vc01zbbxr.tech	TXT	позже, из Certificate Manager
cdn.vc01zbbxr.tech	CNAME	позже, из CDN-ресурса

MX на vc01zbbxr.tech не заводить.

Проверка:

```
dig +short vc01zbbxr.tech A
dig +short www.vc01zbbxr.tech A
dig +short origin.vc01zbbxr.tech A
dig +short www.vc01zbbxr.tech AAAA
dig +short origin.vc01zbbxr.tech AAAA
```

Все три A должны быть **NODE_IP**. Записей **AAAA** не должно быть, пока у ВМ нет рабочего IPv6: Let's Encrypt пойдёт на AAAA и получит timeout.

Группа безопасности Yandex Cloud (сразу, не ждать пункта 13): входящие TCP **22, 80, 443, 8080** с 0.0.0.0/0. HTTP-01 (выпуск сертификата) ходит на порт **80 с интернета**. Если открыт только 443, curl https://vc01zbbxr.tech/ будет 200, а acme.sh на www/origin упадёт с Timeout during connect (likely firewall problem).

5. Установка 3x-ui v3.6.0

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) v3.6.0
x-ui
```

Сохраните логин, пароль, порт **8080** и секретный URI path. Если установщик спрашивает SSL/домен — **пропустите**: сертификат выпускаем в пункте 6 сразу на три имени. WARP не включать. Логи Xray: warning.

```
sudo mkdir -p /etc/systemd/system/x-ui.service.d
sudo nano /etc/systemd/system/x-ui.service.d/limits.conf
```

```
[Service]
LimitNOFILE=1048576
LimitNPROC=65535
```

```
sudo systemctl daemon-reload && sudo systemctl restart x-ui
```

Панель: https://vc01zbbxr.tech:8080/<секретный_путь>.

Подписка (выключена)

Panel Settings → Subscription:

Поле	Значение
Enable Subscription	OFF
Subscription Path	/oob/ (задать даже при OFF)

Ключи выдаются вручную: QR / Copy link / JSON из раздела 17. Порт 2096 в UFW не открывать.

6. Один сертификат ACME панели — панель, nginx, зеркало, origin

Certbot на ВМ **не ставим**. Зх-ui ставит acme.sh и выпускает сертификат Let's Encrypt. Этот же набор файлов используют:

Кто	Зачем
Панель Зх-ui на :8080	HTTPS админки без предупреждения браузера
nginx vc01zbbxr.tech / www	зеркало Debian
nginx origin.vc01zbbxr.tech	HTTPS, которым CDN забирает origin
Инбаунд в Зх-ui	в карточке можно выбрать сертификат панели (тот же ACME)

Клиенты VPN в этой схеме всё равно входят на **cdn.vc01zbbxr.tech**: там свой сертификат из Certificate Manager (пункт 12.3). Сертификат панели нужен на ВМ: браузеру на зеркале, CDN на origin и панели. Если позже сделаете TLS-инбаунд, на который клиент ходит **напрямую** (не через nginx proxy_pass http://), в настройках инбаунда укажите этот же сертификат панели.

Имена в одном сертификате (SAN): **vc01zbbxr.tech**, **www.vc01zbbxr.tech**, **origin.vc01zbbxr.tech**. Имя **cdn.** сюда **не** включать.

Порт **80 должен быть свободен** (nginx ещё не установлен) и **открыт с интернета** в группе безопасности ВМ. А-записи из пункта 4 уже должны указывать на NODE_IP.

```
# acme.sh появляется при первом SSL в меню Зх-ui; если его ещё нет:
curl -s https://get.acme.sh | sh
source ~/.bashrc
mkdir -p /root/cert/vc01zbbxr.tech
```

Либо откройте **х-ui** → пункт SSL Certificate Management → **Get SSL (Domain)** и введите **vc01zbbxr.tech** (так ставится acme.sh и каталог **/root/cert/...**). Меню выпускает **одно** имя — **www** и **origin** допишите командой ниже, даже если apex уже выпущен.

```
~/acme.sh/acme.sh --set-default-ca --server letsencrypt
~/acme.sh/acme.sh --issue --standalone --httpport 80 \
  -d vc01zbbxr.tech \
  -d www.vc01zbbxr.tech \
  -d origin.vc01zbbxr.tech \
  --force

~/acme.sh/acme.sh --install-cert -d vc01zbbxr.tech \
  --fullchain-file /root/cert/vc01zbbxr.tech/fullchain.pem \
  --key-file /root/cert/vc01zbbxr.tech/privkey.pem \
  --reloadcmd "systemctl reload nginx ; x-ui restart"
```

Пока nginx нет, `reload nginx` в hook просто ничего не сделает — это нормально. Когда nginx появится (пункт 9), тот же hook начнёт подхватывать продление.

Привязать файлы к панели (если меню SSL само не прописало пути):

`x-ui` → SSL Certificate Management → **Set Cert paths for the panel**

или в веб-морде **Panel Settings** → **Certificate**:

Поле	Значение
Certificate public key file path	<code>/root/cert/vc01zbbxr.tech/fullchain.pem</code>
Certificate private key file path	<code>/root/cert/vc01zbbxr.tech/privkey.pem</code>

Перезапуск панели: `x-ui restart`.

Обязательная проверка имён в сертификате (меню `3x-ui` выпускает только арех — без трёх `-d` шаг с `origin` позже упадёт):

```
openssl x509 -in /root/cert/vc01zbbxr.tech/fullchain.pem -noout -subject -ext subjectAltName
```

В выводе должны быть **все три**: `vc01zbbxr.tech`, `www.vc01zbbxr.tech`, `origin.vc01zbbxr.tech`. Если `origin` нет — повторите `--issue` с тремя `-d` и `--install-cert` выше. Не идите дальше, пока SAN не полный.

Панель: `https://vc01zbbxr.tech:8080/<секретный_путь>` без ошибки сертификата (имя арех есть в SAN).

Дальше nginx в пунктах 7–9 указывает **на эту же директорию**, отдельный Let's Encrypt для зеркала не выпускаем.

7. Зеркало Debian: каталоги и глобальный nginx

```
sudo apt install -y nginx
sudo rm -f /etc/nginx/sites-enabled/default
sudo mkdir -p /var/cache/nginx/mirror /var/cache/nginx/tmp /var/www/mirror /var/www/acme
sudo chown -R www-data:www-data /var/cache/nginx /var/www/mirror
```

Каталог `/var/www/acme` нужен `acme.sh` для продления (webroot), когда порт 80 займёт nginx. Certbot не ставить.

В `/etc/nginx/nginx.conf` директива `worker_rlimit_nofile` живёт **только в корне файла** (рядом с `worker_processes`), не внутри `http { }`. Остальное — внутри `http { }`.

Корень файла, сразу после `worker_processes auto;`:

```
worker_rlimit_nofile 1048576;
```

Внутри `http { ... }`:

```
proxy_cache_path /var/cache/nginx/mirror levels=1:2 keys_zone=mirror:100m
                  max_size=10g inactive=30d use_temp_path=off;
proxy_temp_path /var/cache/nginx/tmp;

server_tokens off;

map $remote_addr $ip_anon {
    ~(?:P<a>\d+\.\d+\.\d+)\.  "$a.0";
    default                  "0.0.0.0";
}
log_format mirror_anon '$ip_anon - [$time_local] "$request" '
                      '$status $body_bytes_sent cache=$upstream_cache_status';

map $request_method $bad_method { default 1; GET 0; HEAD 0; }

client_header_buffer_size 16k;
large_client_header_buffers 8 64k;
```

Визитка `/var/www/mirror/index.html`:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Debian Mirror</title>
  <style>
    body{font:15px/1.6 system-ui,sans-serif;max-width:720px;margin:40px auto;padding:0
16px;color:#222}
    code{background:#f4f4f4;padding:2px 5px;border-radius:4px}
    h1{font-size:24px}
  </style>
</head>
<body>
  <h1>Debian Package Mirror</h1>
  <p>Public caching mirror of the Debian archive
    (<code>/debian</code>, <code>/debian-security</code>).</p>
  <p>Example <code>sources.list</code> entry:</p>
  <pre><code>deb https://vc01zbbxr.tech/debian stable main contrib non-free-firmware</code></pre>
  <p>Content is fetched from the upstream Debian archive on demand and cached.</p>
</body>
</html>
```

`/var/www/mirror/robots.txt`:

```
User-agent: *
Disallow: /
```

`/var/www/mirror/favicon.ico` — иконка во вкладке браузера. Без файла браузер запросит `/favicon.ico` и получит 404; для маскировки зеркала нужна настоящая иконка Debian:

```
sudo curl -fsS -o /var/www/mirror/favicon.ico https://www.debian.org/favicon.ico
sudo chown www-data:www-data /var/www/mirror/favicon.ico
ls -l /var/www/mirror/favicon.ico
```

nginx уже отдаёт этот файл (`location = /favicon.ico`). После того как заработает HTTPS:

```
curl -I https://vc01zbbxr.tech/favicon.ico
```

Ожидается 200. На этом шаге достаточно, что файл лежит на диске.

Сниппет `/etc/nginx/snippets/debian-mirror.conf`:

```
proxy_pass https://deb.debian.org;
proxy_ssl_server_name on;
proxy_ssl_name deb.debian.org;
proxy_set_header Host deb.debian.org;
proxy_set_header Accept-Encoding "";
proxy_http_version 1.1;
proxy_set_header Connection "";

proxy_cache mirror;
proxy_cache_key $uri;
proxy_cache_lock on;
proxy_cache_use_stale error timeout updating http_500 http_502 http_503 http_504;
proxy_ignore_headers Set-Cookie Cache-Control Expires;
proxy_hide_header Set-Cookie;

add_header X-Cache-Status $upstream_cache_status;
```

8. ACL по префиксам CDN

```
sudo tee /usr/local/bin/cdn-acl-update.sh >/dev/null <<'EOF'
#!/bin/bash
set -e
TMP=$(mktemp)
curl -fsS https://tech.cdn.yandex.net/prefixes/yc.json \
  | python3 -c "import json,sys
d=json.load(sys.stdin)['prefixes']
print('\n'.join('allow %s;' % p for p in d))" > "$TMP"
echo "deny all;" >> "$TMP"
install -m 0644 "$TMP" /etc/nginx/cdn-allow.conf
rm -f "$TMP"
nginx -t && systemctl reload nginx
EOF
sudo chmod +x /usr/local/bin/cdn-acl-update.sh
sudo /usr/local/bin/cdn-acl-update.sh

# файл /etc/cron.d/cdn-acl (не каталог). Команду не разрывать.
sudo tee /etc/cron.d/cdn-acl >/dev/null <<'EOF'
17 4 * * 1 root /usr/local/bin/cdn-acl-update.sh >/dev/null 2>&1
EOF
```


Файл должен существовать **до** `include` в конфиге `nginx`.

9. `nginx`: зеркало, туннель, `origin`

`/etc/nginx/conf.d/00-http.conf`:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    location /.well-known/acme-challenge/ { root /var/www/acme; allow all; }
    location / { return 301 https://$host$request_uri; }
}
```

`/etc/nginx/conf.d/10-mirror.conf`:

Ubuntu 24.04 ставит `nginx` **1.24**: отдельной директивы `http2 on`; нет (она с 1.25.1). HTTP/2 включается параметром `listen`.

```
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name vc01zbbxr.tech www.vc01zbbxr.tech;

    ssl_certificate      /root/cert/vc01zbbxr.tech/fullchain.pem;
    ssl_certificate_key  /root/cert/vc01zbbxr.tech/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    access_log /var/log/nginx/mirror.log mirror_anon;
    server_tokens off;

    location = / { root /var/www/mirror; try_files /index.html =404; }
    location = /robots.txt { root /var/www/mirror; }
    location = /favicon.ico { root /var/www/mirror; access_log off; }

    location ~* \.(deb|udeb|tar\.(gz|xz|bz2|zst)|gz|xz|zst)$ {
        if ($bad_method) { return 405; }
        include /etc/nginx/snippets/debian-mirror.conf;
        proxy_read_timeout 300s;
        proxy_cache_valid 200 206 30d;
    }

    location / {
        if ($bad_method) { return 405; }
        include /etc/nginx/snippets/debian-mirror.conf;
        proxy_read_timeout 60s;
        proxy_cache_valid 200 301 302 5m;
        proxy_cache_valid 404 1m;
    }
}
```

`/etc/nginx/conf.d/20-origin.conf` — источник CDN, туннель, зеркало на остальных путях:

```

server {
    listen 443 ssl http2 default_server;
    listen [::]:443 ssl http2 default_server;
    server_name origin.vc01zbbxr.tech;

    ssl_certificate      /root/cert/vc01zbbxr.tech/fullchain.pem;
    ssl_certificate_key  /root/cert/vc01zbbxr.tech/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    access_log /var/log/nginx/origin.log mirror_anon;
    server_tokens off;

    location /healthcheck.api/ {
        include /etc/nginx/cdn-allow.conf;
        error_page 403 = @mirror_root;

        proxy_pass http://127.0.0.1:10443;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_buffering off;
        proxy_request_buffering off;
        proxy_cache off;
        proxy_read_timeout 300s;
        client_max_body_size 0;
        access_log off;
    }

    location @mirror_root { root /var/www/mirror; try_files /index.html =404; }

    location = / { root /var/www/mirror; try_files /index.html =404; }
    location = /favicon.ico { root /var/www/mirror; access_log off; }

    location ~* \.(deb|udeb|tar\.(gz|xz|bz2|zst)|gz|xz|zst)$ {
        if ($bad_method) { return 405; }
        include /etc/nginx/snippets/debian-mirror.conf;
        proxy_read_timeout 300s;
        proxy_cache_valid 200 206 30d;
    }

    location / {
        if ($bad_method) { return 405; }
        include /etc/nginx/snippets/debian-mirror.conf;
        proxy_read_timeout 60s;
        proxy_cache_valid 200 301 302 5m;
        proxy_cache_valid 404 1m;
    }
}

```

```
sudo nginx -t && sudo systemctl reload nginx
```

Если `nginx -t` пишет `Permission denied` на `/root/cert/` (редко, из-за `ProtectHome` у `systemd`) — скопируйте `fullchain.pem` и `privkey.pem` в `/etc/nginx/ssl/` и поправьте пути. Обычно мастер `nginx` работает от `root` и читает `/root/cert/` без копии.

Переключить продление АСМЕ на webroot (иначе `cron acme.sh` снова займёт порт 80 и поспорится с `nginx`):

```
~/acme.sh/acme.sh --issue --webroot /var/www/acme \
-d vc01zbbxr.tech \
-d www.vc01zbbxr.tech \
-d origin.vc01zbbxr.tech \
--force

~/acme.sh/acme.sh --install-cert -d vc01zbbxr.tech \
--fullchain-file /root/cert/vc01zbbxr.tech/fullchain.pem \
--key-file /root/cert/vc01zbbxr.tech/privkey.pem \
--reloadcmd "systemctl reload nginx ; x-ui restart"
```

`--install-cert` запускаяте **только если** `--issue` закончился успехом (в логе есть `Cert success / «Installing full chain»`, без `Invalid status`). Иначе `acme.sh` снова положит старый сертификат только на `vc01zbbxr.tech`.

Проверка `origin` **до** создания CDN:

```
openssl x509 -in /root/cert/vc01zbbxr.tech/fullchain.pem -noout -ext subjectAltName
curl -I https://origin.vc01zbbxr.tech/
curl -I https://vc01zbbxr.tech/
```

Ожидается 200 и HTML визитки. Документация требует, чтобы источник был доступен из интернета до создания ресурса.

Если `--issue` пишет `Timeout during connect (likely firewall problem)` на `www` или `origin`: Let's Encrypt не смог открыть `http://ЭТО_ИМЯ/.well-known/acme-challenge/...` с интернета. HTTPS на 443 тут ни при чём.

1. Группа безопасности ВМ: входящий **TCP 80** с `0.0.0.0/0` (и 443).
2. На ВМ: `sudo ufw status` — если `active`, должно быть `80/tcp ALLOW`. Если нет: `sudo ufw allow 80/tcp && sudo ufw allow 443/tcp`.
3. DNS: `dig +short www.vc01zbbxr.tech A` и `origin.vc01zbbxr.tech A` = `NODE_IP`; AAAA пусто.
4. С **другого** компьютера или телефона (не с самой ВМ): открыть `http://www.vc01zbbxr.tech/` — должен быть редирект или ответ `nginx`, не вечное кручение.
5. Снова `--issue --webroot` с тремя `-d`. Потом `--install-cert`.

Если curl пишет `no alternative certificate subject name matches ... origin` — в сертификате нет `origin`. `nginx` уже на 80, **не** `--standalone`. Сначала добейтесь успешного `--issue` (SAN с тремя именами), затем `--install-cert` и `sudo systemctl reload nginx`.

10. Инбаунд protocol.1

Inbounds → Add Inbound.

Поле	Значение
Remark / Tag	protocol.1
Protocol	VLESS
Listen / Port	127.0.0.1 / 10443
Network / Mode	xhttp / packet-up
Path	/healthcheck.api/
Host	пусто
Security	none
scMaxBufferedPosts	30
Flow	пусто
Certificate	сертификат панели: /root/cert/vc01zbbxr.tech/fullchain.pem + privkey.pem (выбор в UI 3x-ui). На слушателе Security = none — TLS снимает nginx этими файлами. Поле сертификата нужно, чтобы панель знала тот же ACME; не включайте TLS на этом инбаунде, пока nginx проксирует http://127.0.0.1:10443.

External Proxy (Stream → ON → Add): Address `cdn.vc01zbbxr.tech`, Port `443`, Force TLS.

Save → Restart Xray.

11. Маршрутизация Xray

Порядок сверху вниз:

1. inbound `api` → `api`
2. `geoip:private` → `blocked`
3. `protocol bittorrent` → `blocked`
4. `geoip:ru` / `geosite РФ` → `direct`
5. inbound `protocol.1` → `direct` или тег каскадной ноды

12. Настройка CDN в Yandex Cloud

Порядок по [официальному quickstart](#) и [созданию ресурса](#). Отличия от демо-сценария (там HTTP и IP источника) указаны явно: нам нужны **HTTPS к origin** и **Host = origin-домен**, иначе nginx не попадёт в нужный `server`.

12.1. Перед началом

1. Домен и доступ к DNS (уже есть).
2. Вход в [консоль Yandex Cloud](#).
3. Каталог с ВМ; роль не ниже `cdn.editor`.
4. Платёжный аккаунт активен (при блокировке доступ к CDN останавливается).
5. Источник уже отвечает: `https://origin.vc01zbbxr.tech/` открывается из интернета.
6. Группа безопасности ВМ: входящие TCP **80** и **443** (CDN ходит к источнику только по IPv4).

12.2. Активация CDN-провайдера

Первый заход в сервис **Cloud CDN** в консоли обычно предлагает активировать провайдера в каталоге. Без этого создать ресурс нельзя.

CLI:

```
yc cdn provider list-activated
# если пусто:
yc cdn provider activate --type gcore
yc cdn provider list-activated
```

Тип провайдера смотрите в консоли / --help; после активации список не пустой.

12.3. Сертификат CDN-домена в Certificate Manager

По [документации TLS](#):

1. Сервис **Certificate Manager**, тот же каталог, что будет у CDN-ресурса.
2. Выпустить Let's Encrypt для `cdn.vc01zbbxr.tech`.
3. Проверка прав на домен — **только DNS** (TXT или CNAME на `_acme-challenge.cdn.vc01zbbxr.tech`). HTTP-01 не работает: CDN на `/.well-known/acme-challenge/` отвечает 404.
4. Дождаться статуса «выпущен». Скопировать идентификатор сертификата.

12.4. Создать CDN-ресурс (консоль)

1. **Cloud CDN** → вкладка **CDN-ресурсы** → **Создать ресурс**.
2. Блок **Контент**:
 - **Доступ к контенту** — включить.
 - **Запрос контента** — Из одного источника.
 - **Тип источника** — Сервер.
 - **Доменное имя источника** — `origin.vc01zbbxr.tech`
(в демо quickstart указан публичный IP и HTTP; нам нужен **домен origin и HTTPS**, чтобы совпал виртуальный хост nginx.)
 - **Протокол для источников** — **HTTPS**.
 - **Доменное имя** (раздача) — `cdn.vc01zbbxr.tech`.**Основное имя нельзя изменить после создания.**
3. Блок **Дополнительно**:
 - **Переадресация клиентов** — сначала Не использовать (документация: HTTPS-редирект включают **после** привязки сертификата).
 - **Тип сертификата** — Сертификат из Certificate Manager, выбрать сертификат из шага 12.3.
 - **Заголовок Host** — Свое значение = `origin.vc01zbbxr.tech`.
Документация: значение Host должно совпадать с виртуальным хостом источника.
 - **Перенаправление запросов (Rewrite)** — выкл.
 - **Доступ по защищённому токenu** — выкл.
 - **Доступ по IP-адресам** — выкл.
4. **Продолжить**.
5. Раздел **Кеширование**:
 - **Кеширование в CDN** — **выключить**.
 - **Кеширование в браузере** — **выключить**.

- **Кеширование query-параметров** — Кешировать всё (учитывать все параметры), на случай если кеш когда-либо включат.

- **gzip-сжатие** — выкл.

- **Сегментация больших файлов** — выкл.

6. Продолжить.

7. Раздел **HTTP-заголовки и методы**:

- **Разрешенные методы** — по умолчанию (GET/HEAD). POST не включать: аплинк туннеля идёт GET.

- CORS и скрытие заголовков не трогать.

8. **Создать и продолжить**. Раздел **Дополнительно**: экранирование и выгрузка логов — по желанию, для туннеля не обязательны.

9. Дождаться создания ресурса (**до 15 минут**).

CLI (эквивалент):

```
yc cdn resource create cdn.vc01zbbxr.tech \  
  --origin-custom-source origin.vc01zbbxr.tech \  
  --origin-protocol HTTPS \  
  --cert-manager-ssl-cert-id <ID_СЕРТИФИКАТА>
```

Host и кеш после создания донастройте в консоли, если CLI не выставил их.

После появления сертификата на ресурсе: **Переадресация клиентов** → с HTTP на HTTPS.

Не включать: следование перенаправлениям от источника; POST/PUT/PATCH/DELETE.

12.5. CNAME

1. Страница CDN-ресурса → **Обзор** → **Настройки DNS**.

2. Скопировать имя вида e1b83ae3*****.topology.gslb.yccdn.ru.

3. В DNS-хостинге:

```
cdn CNAME e1b83ae3*****.topology.gslb.yccdn.ru.
```

Не использовать ANAME. Документация: при ANAME ответ не зависит от геолокации клиента.

12.6. Проверка CDN

Дождаться обновления DNS (часы).

```
dig +short cdn.vc01zbbxr.tech CNAME  
# должен указать на *.topology.gslb.yccdn.ru  
  
dig +short cdn.vc01zbbxr.tech A  
# сверить IP с https://tech.cdn.yandex.net/prefixes/yc.json  
  
curl -I https://cdn.vc01zbbxr.tech/  
# визитка зеркала  
  
curl -i https://cdn.vc01zbbxr.tech/healthcheck.api/ | head -20  
# ответ Xray, не страница 404 сайта
```

Без запросов **90 дней** ресурс переходит в Not active.

13. Firewall

До этого UFW был **выключен** (пункт 2). Сейчас включаем с нужными портами. Сначала правила, потом enable — иначе можно отрезать SSH.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 80/tcp comment 'ACME + redirect'
sudo ufw allow 443/tcp comment 'nginx TLS'
sudo ufw allow 8080/tcp comment '3x-ui panel'
sudo ufw --force enable
sudo ufw status numbered
```

Порт **2096 не открывать**. В security group VM: TCP 22, 80, 443, 8080.

14. Обслуживание кеша зеркала (раз в неделю)

```
sudo tee /usr/local/bin/mirror-cache-weekly.sh >/dev/null <<'EOF'
#!/bin/bash
set -e
CACHE=/var/cache/nginx/mirror
find "$CACHE" -type f -atime +7 -delete 2>/dev/null || true
find "$CACHE" -mindepth 1 -type d -empty -delete 2>/dev/null || true
USE=$(df --output=pcent "$CACHE" | tail -1 | tr -dc '0-9')
if [ "${USE:-0}" -ge 90 ]; then
    rm -rf "${CACHE:?}/*"
    logger -t mirror "cache disk ${USE}% -> flushed"
fi
systemctl reload nginx
EOF
sudo chmod +x /usr/local/bin/mirror-cache-weekly.sh
sudo tee /etc/cron.d/mirror-cache >/dev/null <<'EOF'
30 4 * * 1 root /usr/local/bin/mirror-cache-weekly.sh >/dev/null 2>&1
EOF
```

15. Выдача ключей (без подписки)

Ключ из панели (после External Proxy) или JSON из раздела 17.

Не менять: `sni=cdn.vc01zbbxr.tech`, `type=xhttp`, `path=/healthcheck.api/`, порт `443`.

16. Проверка ноды

```
ss -tlnp | grep -E ':80|:443|:8080|:10443'
sudo nginx -t
tail -1 /etc/nginx/cdn-allow.conf

curl -sI https://vc01zbbxr.tech/debian/dists/stable/Release | grep -i x-cache-status
curl -Ik https://NODE_IP/
curl -sI https://origin.vc01zbbxr.tech/healthcheck.api/

/usr/local/x-ui/bin/xray-linux-amd64 run -test -config /usr/local/x-ui/bin/config.json
```

Чеклист

- [] UFW выключен в начале (п. 2), в п. 13 включён: 22, 80, 443, **8080**; без 2096
- [] `vpnXX`, `PermitRootLogin no`
- [] `sysctl / limits / BBR`
- [] A-записи зеркала и origin
- [] SAN ACME панели на 3 имени → `/root/cert/vc01zbbxr.tech/`
- [] `nginx ssl_*` указывает на сертификат панели (не certbot)
- [] визитка, `favicon.ico` с `debian.org`, кеш зеркала
- [] `cdn-allow.conf` + cron ACL
- [] инбаунд **protocol.1**, `xhttp`, `packet-up`, `127.0.0.1:10443`
- [] External Proxy: `cdn.vc01zbbxr.tech:443`
- [] подписка **OFF**, путь `/oob/`
- [] провайдер CDN активирован
- [] сертификат CDN в Certificate Manager (DNS)
- [] ресурс: источник `origin....`, HTTPS, Host = origin, кеш выкл.
- [] CNAME `cdn.` → `*.topology.gslb.yccdn.ru`, IP в `yc.json`
- [] клиент: `fingerprint edge`, `alpn []`, GET-uplink, XMUX
- [] недельный cron кеша зеркала

17. Клиентский профиль (фрагмент транспорта)

```
{
  "network": "xhttp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "cdn.vc01zbbxr.tech",
    "alpn": [],
    "fingerprint": "edge",
    "allowInsecure": false
  },
  "xhttpSettings": {
    "host": "cdn.vc01zbbxr.tech",
    "path": "/healthcheck.api/",
    "mode": "packet-up",
    "scMaxEachPostBytes": 1000000,
    "scMinPostsIntervalMs": 30,
    "scMaxConcurrentPosts": 10,
    "extra": {
      "uplinkHTTPMethod": "GET",
      "xmux": {
        "maxConcurrency": "16-32",
        "maxConnections": 0,
        "cMaxReuseTimes": 100,
        "cMaxLifetimeMs": 300000,
        "hKeepAlivePeriod": 30
      }
    }
  },
  "sockopt": {
    "domainStrategy": "UseIPv4",
    "tcpKeepAliveIdle": 60,
    "tcpKeepAliveInterval": 20,
    "tcpUserTimeout": 60000
  }
}
```

DNS: `full:cdn.vc01zbbxr.tech` через `77.88.8.8`. Routing: `udp/443 → block`, `РФ → direct`.

18. Остаточные риски

Оператор CDN видит пути и IP в открытом виде. Кеш CDN нельзя включать. Ширма-зеркало не делает туннель невидимым, но совпадает с ним по рисунку GET. Панель на `:8080` видна в сканерах портов (Shodan) — секретный путь снижает риск, но не убирает порт из интернета.

Изменения этой редакции: в **пункте 2** UFW выключается до конца установки, правила — в пункте 13. **Один АСМЕ 3x-ui** на `vc01zbbxr.tech` + `www` + `origin`; nginx читает `/root/cert/vc01zbbxr.tech/`. `worker_rlimit_nofile` в корне `nginx.conf`. HTTP/2: `listen 443 ssl http2`. HTTP-01: порт 80 в группе безопасности облака; `--install-cert` только после успешного `--issue`.